IHMS Emergency

راهنمای بسته‌بندی و انتشار اپلیکیشن اندروید

این سند مسیر کامل تبدیل سامانه هوشمند اطلاع‌رسانی شرایط اضطراری HSE به یک اپلیکیشن اندروید قابل انتشار را گام‌به‌گام شرح می‌دهد؛ از بسته‌بندی رسمی با Capacitor و روش سریع PWA Builder تا امضای نسخه نهایی، افزودن اعلان‌های Push بومی با Firebase و استقرار سرور تولیدی با اتصال HTTPS و سرویس پایدار.

فایل‌های آماده‌شده در پروژه:

`capacitor.config.ts` `scripts/build-android.sh`

نسخه سند: ۱٫۰ · **تاریخ:** آبان ۱۴۰۵ · **سازمان نمونه:** زرین برگ پارس

SECTION 01

۱. نمای کلی معماری و فایل‌های آماده

پیش از شروع بسته‌بندی، درک صحیح از معماری سامانه ضروری است. اپلیکیشن IHMS Emergency یک سامانهٔ تمام‌عیار اطلاع‌رسانی لحظه‌ای است که از سه جزء اصلی تشکیل شده: وب‌اپلیکیشن Next.js (رابط کاربری و API)، سرویس سوکت مستقل Socket.IO روی پورت `3003` برای ارسال هشدار در لحظه، و پایگاه‌دادهٔ SQLite. به همین دلیل فایل APK صرفاً یک **«پوستهٔ» (Shell)** اندرویدی است که رابط کاربری را در WebView بارگذاری می‌کند و هوش سامانه — پردازش هشدارها، نگاشت گروه‌های هدف، تأییدیه‌ها و تاریخچه — همگی روی سرور میزبان اجرا می‌شود.

این طراحی یک مزیت عملیاتی مهم دارد: هر بار که نسخهٔ جدیدی از رابط کاربری یا منطق اطلاع‌رسانی منتشر کنید، کاربران **بدون نیاز به نصب به‌روزرسانی جدید** نسخهٔ تازه را دریافت می‌کنند، چرا که APK فقط محتوا را از سرور بارگذاری می‌کند. در مقابل، تنها اجزایی که روی خود دستگاه اجرا می‌شوند، پخش صدای آژیر (Web Audio)، ویبره (Vibration API) و نمایش صفحهٔ قرمز تمام‌صفحه است که در WebView به‌طور کامل پشتیبانی می‌شود. بنابراین پیش از توزیع APK، استقرار سرور با گواهی HTTPS (بخش ۶) الزامی است.



DIAGRAM 1 — معماری سه‌لایهٔ سامانه (کلاینت → سرور → داده)

برای تسریع فرایند، دو فایل کلیدی از قبل در پروژه آماده شده است. فایل `capacitor.config.ts` پیکربندی کامل Capacitor شامل شناسهٔ پکیج، عنوان اپ، رنگ اسپلش‌اسکرین و آدرس سرور را در خود دارد و فایل `scripts/build-android.sh` تمام مراحل نصب پکیج‌ها، ساخت نسخهٔ production، افزودن پلتفرم اندروید و کامپایل Gradle را در یک دستور جمع می‌کند.

> **نکتهٔ معماری — دو مسیر توزیع موازی**
>
> این پروژه هم‌زمان یک **PWA کامل** است (با manifest و Service Worker). یعنی حتی بدون APK، کاربران اندروید می‌توانند از منوی مرورگر گزینهٔ «Add to Home Screen» را انتخاب کنند و اپ به شکل مستقل با آیکون اختصاصی نصب شود. APK وقتی ارزش افزوده پیدا می‌کند که به **اعلان‌های Push بومی** (بخش ۵)، انتشار در کافه‌بازار یا گوگل‌پلی و کنترل کامل روی اسپلش‌اسکرین نیاز داشته باشید.

۲. پیش‌نیازهای محیط ساخت

ساخت APK به یک محیط توسعهٔ اندروید نیاز دارد که یک‌بار روی سیستم مهندس راه‌اندازی می‌شود و پس از آن، ساخت هر نسخهٔ جدید فقط چند ثانیه طول می‌کشد. جدول زیر اجزای الزامی به همراه نسخهٔ پیشنهادی را نشان می‌دهد. توصیه می‌شود دقیقاً همین نسخه‌ها را نصب کنید؛ انحراف از نسخهٔ JDK به‌ویژه رایج‌ترین علت خطاهای Gradle است.

جزء	نسخهٔ پیشنهادی	توضیح
Node.js یا Bun	Node 20+ / Bun 1.x	اجرای پروژهٔ Next.js و اسکریپت‌های ساخت
JDK	Temurin / OpenJDK 17	الزام Gradle 8 و Android SDK؛ نسخهٔ ۱۷ دقیقاً الزامی است
Android SDK	Build-Tools 34 + API 34	از طریق Android Studio یا فقط ابزار خط فرمان (cmdline-tools)
Android Studio	Latest	اختیاری اما توصیه‌شده؛ شبیه‌ساز و SDK Manager را فراهم می‌کند
adb	Platform-Tools	نصب مستقیم APK روی دستگاه متصل برای تست

پس از نصب، متغیرهای محیطی زیر را در فایل `~/.bashrc` یا `~/.zshrc` تنظیم کنید تا Gradle بتواند JDK و SDK را پیدا کند. اگر Android Studio را نصب کرده باشید، SDK به‌طور پیش‌فرض در مسیر `~/Android/Sdk` قرار می‌گیرد:

```
# ~/.bashrc — Android build environment
export JAVA_HOME=/usr/lib/jvm/temurin-17-jdk-amd64
export ANDROID_HOME=$HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin
export PATH=$PATH:$ANDROID_HOME/platform-tools

# پذیرش لایسنس‌ها (بار اول الزامی است)
yes | sdkmanager --licenses

# بررسی صحت نصب
java -version   && sdkmanager --version
```

هشدار رایج — ناسازگاری JDK

اگر پیام خطایی مشابه `Unsupported class file major version` یا `Could not determine java version` دیدید، تقریباً همیشه علت اجرای Gradle با JDK نامناسب (مثلاً ۱۱ یا ۲۱) است. با دستور `gradle -version` یا متغیر `JAVA_HOME` مطمئن شوید دقیقاً JDK 17 در استفاده است.

۳. مسیر الف: بسته‌بندی رسمی با Capacitor

Capacitor روش رسمی و توصیه‌شده برای تبدیل این وب‌اپ به اپ اندروید است؛ خروجی آن یک پروژهٔ اندروید استاندارد و کاملاً قابل سفارشی‌سازی است که در آن به منابع بومی، افزونه‌های Push و تنظیمات شبکه دسترسی مستقیم دارید. کل فرایند در اسکریپت `scripts/build-android.sh` خودکارسازی شده و شامل شش مرحله است: نصب پکیج‌های Capacitor، بررسی پیکربندی، ساخت نسخهٔ production وب، افزودن پلتفرم اندروید، همگام‌سازی فایل‌ها و در نهایت کامپایل Gradle.

۳-۱. تنظیم آدرس سرور

فایل `capacitor.config.ts` را باز کنید و مقدار `server.url` را با آدرس دامنه‌ای که سرور روی آن مستقر شده (بخش ۶) جایگزین کنید. استفاده از HTTPS الزامی است؛ در غیر این صورت اندروید درخواست‌ها را به‌عنوان ترافیک ناامن مسدود می‌کند. برای تست لوکال روی دستگاه فیزیکی در همان شبکه، می‌توانید موقتاً آدرس IP سیستم خود را با `cleartext: true` قرار دهید:

```
// capacitor.config.ts
server: {
  url: 'https://ihms.YOUR-DOMAIN.com',  // ← دامنهٔ خودتان
  cleartext: false,
}
```

۳-۲. ساخت APK نسخهٔ تست (Debug)

حالا اسکریپت آماده را اجرا کنید. بار اول چند دقیقه طول می‌کشد چون Gradle وابستگی‌ها را دانلود می‌کند؛ اجراهای بعدی بسیار سریع‌تر خواهند بود. خروجی در مسیر `android/app/build/outputs/apk/debug/app-debug.apk` ذخیره می‌شود:

```
cd /path/to/ihms-emergency
bash scripts/build-android.sh          # ساخت debug

# نصب روی دستگاه متصل (USB debugging فعال باشد)
adb install -r android/app/build/outputs/apk/debug/app-debug.apk
```

اگر دسترسی به کابل ندارید، فایل APK را به هر روشی به گوشی منتقل کنید؛ هنگام نصب، اندروید از شما اجازهٔ «نصب از منابع ناشناس» می‌خواهد که برای تست داخلی کاملاً طبیعی است. پس از نصب، اپ را باز کنید — باید صفحهٔ ورود IHMS Emergency با فونت وزیرمتن و دکمهٔ قرمز فعال‌سازی بلافاصله نمایش داده شود. برای اطمینان از سلامت اتصال لحظه‌ای، نشان «لحظه‌ای» در هدر باید فعال باشد.

۳-۳. امضای نسخهٔ Release و ساخت AAB

نسخهٔ Debug فقط برای تست است و برای توزیع رسمی (کافه‌بازار، گوگل‌پلی یا توزیع سازمانی) باید با یک کلید رمزنگاری‌شده امضا شود. اول یک Keystore اختصاصی بسازید و آن را **همیشه** حفظ کنید — بدون همین فایل، هیچ‌وقت نمی‌توانید به‌روزرسانی برای همان اپ منتشر کنید:

```
# ساخت keystore (یک‌بار برای همیشه) امضا
keytool -genkey -v -keystore ihms-release.keystore \
  -alias ihms -keyalg RSA -keysize 2048 -validity 10000

# android/key.properties
storePassword=****  keyPassword=****
keyAlias=ihms
storeFile=../../ihms-release.keystore
```

سپس بلوک `signingConfigs` را در فایل `android/app/build.gradle` اضافه کرده و به buildType مربوط به release متصل کنید. جزئیات کامل این قطعه‌کد در فایل‌های راهنمای رسمی Capacitor موجود است و پس از آن، دو خروجی در دسترس خواهید داشت: فایل `app-release.apk` برای توزیع مستقیم و فایل `app-release.aab` (با دستور `./gradlew bundleRelease`) که فرمت الزامی گوگل‌پلی است. برای کافه‌بازار نسخهٔ APK کافی است.

تجربهٔ پیشنهادی تست نهایی

پیش از توزیع گسترده، سناریوی کامل اضطراری را روی دستگاه واقعی اجرا کنید: با حساب `dispatcher` یک حریق فعال کنید، روی گوشی دوم با حساب `worker1` صفحهٔ قرمز و صدای آژیر را ببینید، هشدار را تأیید کنید و سپس پایان شرایط اضطراری را از پنل پایش صادر کنید. این همان سناریویی است که در محیط توسعه به‌طور کامل تست شده است.

SECTION 04

۴. مسیر ب: روش سریع PWA Builder

اگر نمی‌خواهید محیط اندروید راه‌اندازی کنید یا فقط به‌سرعت به یک فایل نصبی برای آزمون مدیریتی نیاز دارید، سرویس رایگان PWA Builder (محصول مایکروسافت) می‌تواند از روی نسخهٔ مستقرشدهٔ PWA، بدون هیچ ابزار محلی، فایل APK و AAB امضا شده تولید کند. پیش‌نیاز این روش، استقرار سرور با HTTPS معتبر است (بخش ۶)، چون این سرویس manifest و Service Worker اپ شما را از روی دامنهٔ عمومی می‌خواند.

۱ **استقرار سرور با HTTPS** — مطابق بخش ۶ سامانه را روی دامنهٔ نهایی مستقر کنید و با مرورگر گوشی یک‌بار وارد اپ شوید تا Service Worker ثبت شود.

۲ **بررسی در pwabuilder.com** — آدرس دامنه را وارد کنید؛ امیدوارکننده است که نمرات Manifest و Service Worker این پروژه از قبل بهینه شده‌اند.

۳ **Package for stores → Android** — گزینهٔ Android را انتخاب و Hosted URL را همان دامنهٔ خود بگذارید؛ Package ID را مطابق `ir.zarinpars.ihms.emergency` تنظیم کنید.

۴ **دانلود و امضا** — بستهٔ zip شامل APK/AAB دریافت می‌شود؛ برای توزیع رسمی مطابق بخش ۳-۳ با Keystore خود امضا کنید.

مزیت اصلی این مسیر سرعت و صفر بودن نیاز به ابزار محلی است؛ اما محدودیت‌هایی هم دارد: کنترل کمتری روی اسپلش‌اسکرین و آیکون‌های تطبیقی (Adaptive Icons) دارید، افزودن افزونه‌های بومی مانند Push Notification

فیسبوک (بخش ۵) در این مسیر ممکن نیست و هر تغییر پیکربندی نیاز به تکرار فرایند در سایت دارد. بنابراین توصیهٔ نهایی ما این است که PWA Builder را برای «نمونهٔ سریع مدیریت» و Capacitor را برای «نسخهٔ نهایی سازمانی» استفاده کنید.

SECTION 05

۵. اعلان Push بومی با Firebase (FCM)

در معماری فعلی، هشدار اضطراری از طریق سوکت لحظه‌ای منتقل می‌شود که وقتی اپ باز یا در پس‌زمینهٔ اخیر است، بی‌نقص کار می‌کند و صفحهٔ قرمز بلافاصله ظاهر می‌شود. اما اگر کاربر اپ را کاملاً بسته باشد یا دستگاه در حالت عمیق صرفه‌جویی باشد، سوکت قطع است و پیام تا باز شدن بعدی اپ تحویل نمی‌شود. برای پوشش این شکاف — که در سامانه‌های اضطراری حیاتی است — اعلان‌های Push بومی Android از طریق سرویس Firebase Cloud Messaging (FCM) به معماری اضافه می‌شود:



DIAGRAM 2 — توزیع دوکانالهٔ هشدار: سوکت برای اپ باز، FCM برای اپ بسته

۱ **ساخت پروژهٔ Firebase** — در کنسول Firebase پروژه بسازید، اپ اندرویدی با Package ID دقیق `ir.zarinpars.ihms.emergency` ثبت و فایل `google-services.json` را در پوشهٔ `android/app/` قرار دهید.

۲ **نصب افزونهٔ پیام‌رسان** — پکیج `@capacitor-firebase/messaging` را نصب و در جریان ورود کاربر، توکن FCM را دریافت و به‌همراه شناسهٔ کاربر در جدول کاربران ذخیره کنید.

۳ **ارسال سمت سرور** — در مسیر `POST /api/alerts` پس از emit سوکت، با پکیج `firebase-admin` برای توکن‌های همهٔ گیرندگان، اعلان با عنوان حادثه و صدای آژیر پیش‌فرض ارسال کنید.

۴ **انتخاب آواتار اعلان** — در اندروید ۱۳+ مجوز `POST_NOTIFICATIONS` در زمان اجرا درخواست می‌شود؛ افزونهٔ فوق این جریان را مدیریت می‌کند.

```
// افزودن به سمت سرور — src/app/api/alerts/route.ts
import admin from 'firebase-admin'

// پس از emitToRoom(...) فعال بدون سوکت گیرندگانِ برای:
await admin.messaging().sendEachForMulticast({
  tokens: offlineTargetTokens,            // توکن‌های FCM گیرندگان
  notification: {
    title: `🚨 ${title}`,
    body: `${location} — ${INCIDENT_TYPES[type].label}`,
  },
  android: { priority: 'high', notification: { channelId: 'ihms-emergency', sound: 'siren'
} },
})
```

نکته — پوشش iOS

اگر در آینده نسخهٔ iOS هم مدنظر بود، همین زیرساخت FCM از طریق APNs پشتیبانی می‌شود؛ تنها افزوده، ساخت اپ iOS در Xcode و بارگذاری کلید APNs در کنسول Firebase است. صفحهٔ قرمز و آژیر وب‌اپ نیز در Safari/iOS با محدودیت‌های autoplay صدا (نیازمند اولین تعامل کاربر) کار می‌کند.

SECTION 06

۶. استقرار سرور تولیدی

سرور، قلب تپندهٔ سامانه است و پایداری آن به‌معنای قابلیت اطمینان کل سامانهٔ اضطراری است. حداقل زیرساخت پیشنهادی یک VPS با ۲ هستهٔ CPU و ۴ گیگابایت RAM است که توزیع‌های اوبونتو ۲۲ به بالا به‌خوبی پاسخگو هستند. پروسهٔ استقرار شامل نصب رانتایم، ساخت نسخهٔ production، اجرای دو فرایند (وب و سوکت) زیر نظر systemd و قرار دادن یک Reverse Proxy با HTTPS جلوی آن‌هاست.

۱ **انتقال کد و نصب وابستگی‌ها** — ریپازیتوری را روی سرور کلون کرده و با `bun install` وابستگی‌ها را نصب و دیتابیس را با `bun run db:push` و Seed اولیه راه‌اندازی کنید.

۲ **ساخت و اجرا** — با `bun run build` نسخهٔ production ساخته و سپس `bun run start` را اجرا کنید؛ سرویس سوکت را هم با `bun --hot index.ts` در پوشهٔ mini-services/emergency-socket بالا بیاورید.

۳ **Reverse Proxy و HTTPS** — Caddy یا Nginx را جلوی هر دو سرویس بگذارید؛ Caddy به‌طور خودکار گواهی Let's Encrypt صادر و تمدید می‌کند.

۴ **مدیریت فرایندها** — هر دو سرویس را به‌صورت systemd unit تعریف کنید تا پس از ریبوت، خودکار بالا بیایند.

```
# /etc/caddy/Caddyfile — پراکسی وب + سوکت
ihms.YOUR-DOMAIN.com {
    # وب‌اپ Next.js
    handle {
        reverse_proxy localhost:3000
    }
    # مسیر سوکت لحظه‌ای
    handle /socket.io* {
        reverse_proxy localhost:3003
    }
}
```

```
# /etc/systemd/system/ihms.service — سرویس وب
[Unit]
Description=IHMS Emergency Web
After=network.target

[Service]
WorkingDirectory=/opt/ihms
ExecStart=/usr/local/bin/bun run start
Environment=NODE_ENV=production
Environment=IHMS_SECRET=یک-رشته-تصادفی-قوی
Restart=always

[Install]
WantedBy=multi-user.target
```

دو نکتهٔ حیاتی امنیتی

اول اینکه متغیر `IHMS_SECRET` که امضای توکن سوکت‌ها را بر عهده دارد باید در سرور مقدار قوی و یکتا بگیرد و بین سرویس وب و سرویس سوکت یکسان باشد. دوم اینکه فایل دیتابیس `db/custom.db` تنها نسخهٔ ذخیرهٔ کاربران، گروه‌ها و تاریخچهٔ حوادث است — یک cron شبانه برای بکاپ آن (مثلاً به S3 یا دیسک دوم) تعریف کنید.

SECTION 07

۷. عیب‌یابی و چک‌لیست انتشار

اکثر مشکلات گزارش‌شده در بسته‌بندی و استقرار این نوع سامانه‌ها الگوهای تکراری دارند. جدول زیر پرتکرارترین آن‌ها و راه‌حل قطعی هر کدام را جمع‌بندی می‌کند؛ پیش از هر جست‌وجوی جدید، اول این جدول را ببینید.

علامت	علت محتمل	راه‌حل
Gradle خطای نسخهٔ Java می‌دهد	JDK غیر از ۱۷	تنظیم `JAVA_HOME` روی JDK 17 و اجرای مجدد
اپ باز می‌شود ولی «قطع» می‌دهد	سوکت از پشت پراکسی رد نمی‌شود	مسیر `/socket.io*` در Caddy/Nginx به پورت 3003 فوروارد شود + Upgrade headers
صفحهٔ سفید در WebView	server.url اشتباه یا HTTPS نامعتبر	بررسی دامنه در مرورگر گوشی؛ گواهی معتبر Let's Encrypt نصب شود
صدای آژیر پخش نمی‌شود	سیاست autoplay مرورگر	اولین تعامل کاربر (لمس دکمهٔ ورود) صدا را unlock می‌کند؛ در APK WebView این محدودیت کمتر است
هشدار در اپ بسته نمی‌رسد	سوکت نیازمند اپ باز است	`پیاده‌سازی` FCM طبق بخش ۵
اعلان روی گوشی شیائومی/سامسونگ نمی‌آید	بهینه‌سازی باتری	اپ را از Battery Optimization مستثنا و Auto-start را فعال کنید
خطای License not accepted	لایسنس‌های SDK پذیرفته نشده	اجرای `yes | sdkmanager --licenses`

چک‌لیست نهایی پیش از توزیع

- ✅ سرور روی دامنهٔ نهایی با HTTPS معتبر مستقر شده و نشان «لحظه‌ای» در اپ فعال است
- ✅ سناریوی کامل اضطراری (فعال‌سازی ← صفحهٔ قرمز ← تأیید ← پایش ← پایان) روی دو گوشی واقعی تست شده
- ✅ مقادیر `IHMS_SECRET` و رمزهای حساب admin از مقادیر نمونه تغییر کرده‌اند
- ✅ کدهای فعال‌سازی اولیه باطل و کدهای واقعی پرسنل صادر شده‌اند
- ✅ APK با Keystore سازمان امضا و فایل Keystore در محل امن ذخیره شده است
- ✅ بکاپ خودکار شبانهٔ دیتابیس و مانیتورینگ سرویس‌ها (systemd + لاگ) فعال است
- ✅ اشتراک سازمان در پنل مدیریت با تاریخ پایان واقعی تنظیم شده است

پس از تیک خوردن تمام موارد بالا، سامانهٔ شما از نظر فنی آمادهٔ استقرار در محیط واقعی است. توصیه می‌شود انتشار را به‌صورت مرحله‌ای انجام دهید: ابتدا یک پروژهٔ پایلوت با تیم HSE، سپس توسعهٔ آن به سایر واحدها. بازخوردهای همین فاز پایلوت — از سرعت رسیدن هشدار تا بلندی صدای آژیر در محیط پر سروصدا — بهترین ورودی برای تنظیمات نهایی خواهد بود.

READY FOR PUBLICATION

اپلیکیشن شما آمادهٔ انتشار است

با اجرای اسکریپت آمادهٔ پروژه و رعایت چک‌لیست این راهنما، نسخهٔ اندروید IHMS Emergency در کمتر از یک ساعت قابل ساخت و امضا است. به یاد داشته باشید که قلب سامانه — سرور اطلاع‌رسانی لحظه‌ای — باید پیش از توزیع APK روی زیرساخت پایدار با گواهی HTTPS مستقر شود.

حساب‌های نمونه برای آزمون نهایی:

مدیر سامانه: admin / admin123

فعال‌ساز (کارشناس HSE): dispatcher / 123456

پرسنل (گیرنده هشدار): worker1 / 123456

ورود با کد فعال‌سازی: HSE-CODE-01